Pulse Width Modulation (PWM)

From Signals to Timer Outputs

Professor: Nerio Andrés Montoya
namontoy@unal.edu.co

2026-S2

Universidad Nacional de Colombia

Agenda

Signals

PWM Fundamentals

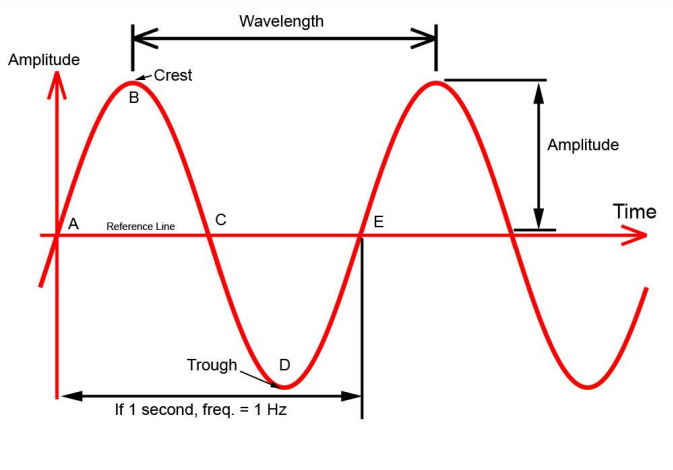
PWM in the STM32 Timer

Configuring PWM in C

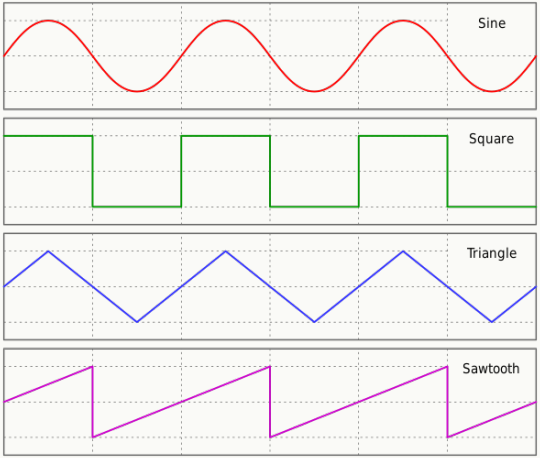
Closing

Signals

Simple Sine Wave

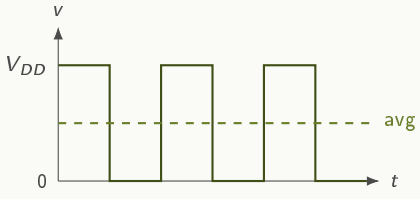


Other Types of Signals



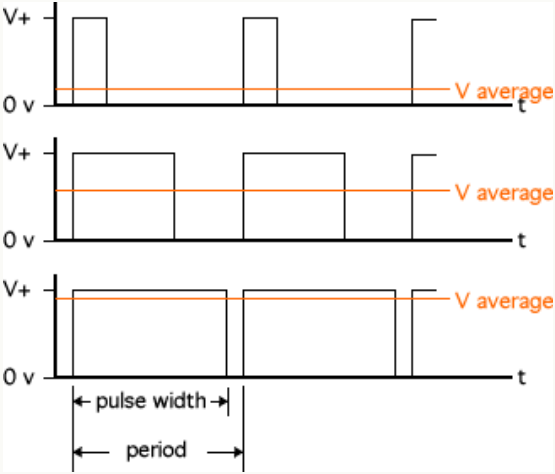
Analog vs. Digital Outputs

- A GPIO output has only **two levels**: 0 V and V_{DD} (3.3 V)
- Many loads need an *in-between* value: LED brightness, motor speed, heater power
- The MCU on our Nucleo has **no DAC** on the F411 (no analog output pin)
- Idea: switch very fast between 0 and V_{DD} and let the load *average* the signal

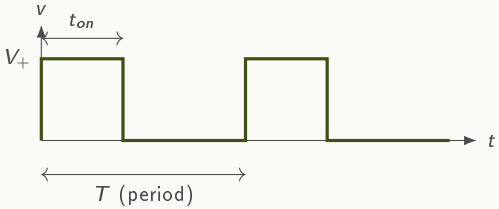


PWM Fundamentals

Schematic of a PWM Signal



Period, Frequency and Duty Cycle

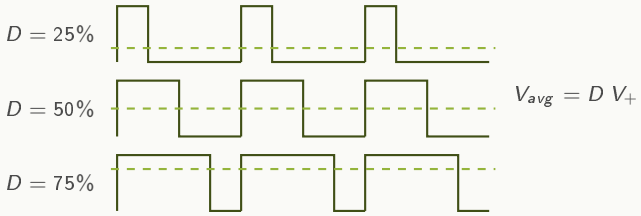


$$f = \frac{1}{T}, \qquad D = \frac{t_{on}}{T}$$

$$V_{avg} = D \cdot V_+$$

- D ranges from 0 to 100 %; the frequency stays **fixed**, only the pulse width changes
- $D = 0\,\%$: always low $\quad D = 100\,\%$: always high
- Example: $V_+ = 3.3$ V, $D = 25\,\% \Rightarrow V_{avg} = 0.83$ V

Duty Cycle → Average Value



- The dashed line is the **average** value
- A load with "inertia" (inductance, thermal mass, the eye, mechanics) responds to the average, not to each pulse
- PWM frequency must be *much faster* than the load's response

Why PWM and Not a Voltage Divider or Linear Regulator?

Linear (resistive) control

The pass element drops $V_{in} - V_{out}$ at the load current.

12 V $\rightarrow$ 3 V at 1 A:
$P = (12 - 3) \times 1 = 9$ W burned as heat

PWM (switching) control

The switch is either fully ON (low V_{DS}) or fully OFF (zero current).

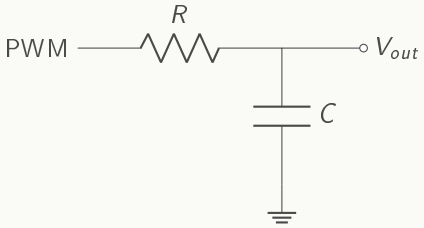
Ideal loss ≈ 0; real loss = conduction + switching losses (a few %)

- This is why PWM is the standard way to control power with an MCU
- Cost: ripple and electrical noise at the switching frequency

Where Do We Use PWM?

- **LED dimming:** $f > 100$ Hz so the eye sees no flicker
- **DC motor speed:** 10–30 kHz (above hearing) $\rightarrow$ see the motors deck
- **Hobby servo:** 50 Hz, pulse width 1–2 ms sets the angle
- **Heaters:** slow PWM (seconds) on a thermal load
- **Switching power supplies** (buck/boost converters)
- **Simple DAC:** PWM + RC low-pass filter $\rightarrow$ analog voltage
- **Buzzers, audio tones**

PWM as a DAC: the RC Low-Pass Filter

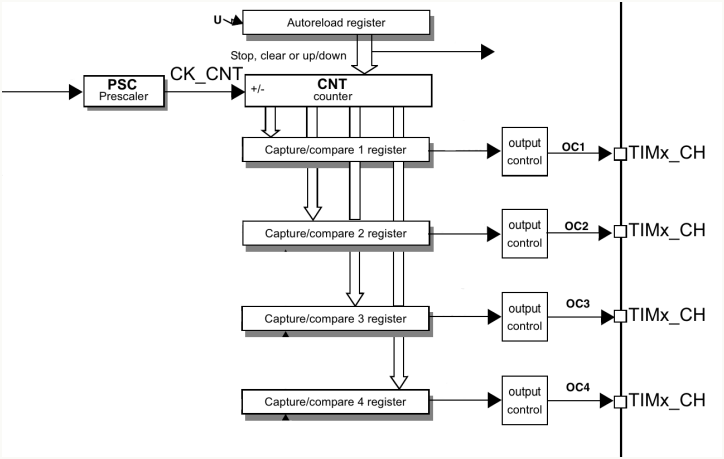


$$f_c = \frac{1}{2\pi RC} \ll f_{PWM}$$

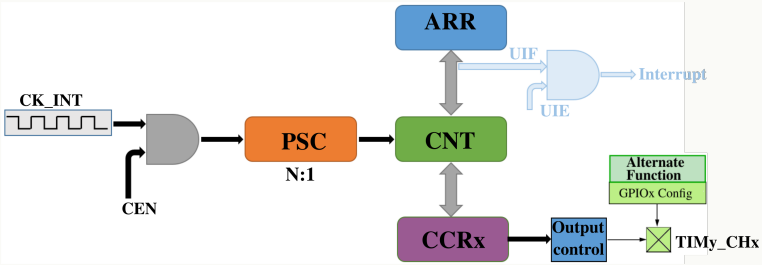
- The filter removes the pulses and keeps the average: $V_{out} \approx D\, V_{DD}$
- Lower $f_c \Rightarrow$ less ripple but slower response
- Example: $R = 10$ kΩ, $C = 1$ μF $\Rightarrow f_c \approx 16$ Hz for $f_{PWM} = 20$ kHz

PWM in the STM32 Timer

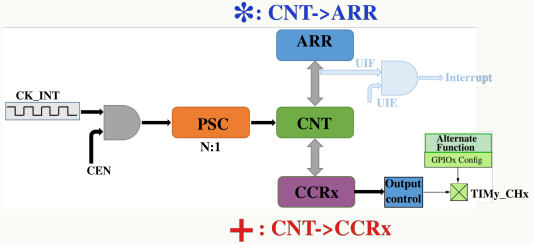
PWM Module (STM32)

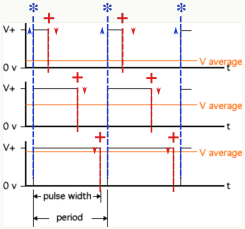


PWM Module — Taller V Version

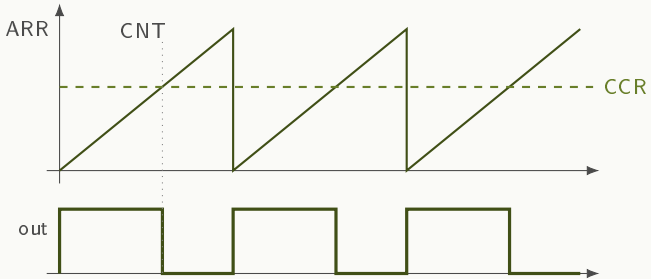


PWM Module and Signal





How the Timer Builds the PWM Signal



- The counter **CNT** counts from 0 to **ARR**, then restarts
- In PWM mode 1 the output is high while $\mathrm{CNT} < \mathrm{CCR}$
- ARR sets the **period**; CCR sets the **pulse width**
- *Mode 2* is the inverse (low while $\mathrm{CNT} < \mathrm{CCR}$)

PWM Frequency and Duty: the Equations

$$f_{CK_CNT} = \frac{f_{CK_INT}}{PSC + 1}$$

$$f_{PWM} = \frac{f_{CK_CNT}}{ARR + 1}, \qquad D = \frac{CCR}{ARR + 1}$$

Example (16 MHz timer clock)

$PSC = 15 \Rightarrow f_{CK_CNT} = 1$ MHz
$ARR = 999 \Rightarrow f_{PWM} = 1$ kHz
$CCR = 250 \Rightarrow D = 25\,\%$

- **PSC** divides the clock
- **ARR** sets the period
- **CCRx** sets the duty cycle of channel x
- One timer has one ARR: all its channels share the *same frequency* but each has its own duty

Frequency vs. Resolution

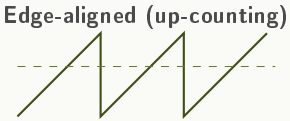
- The number of duty steps is $ARR + 1$: **resolution** $= \log_2(ARR + 1)$ **bits**
- At a fixed timer clock, higher PWM frequency means fewer duty steps:

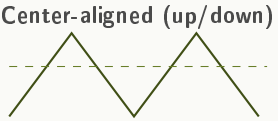
$$ARR + 1 = \frac{f_{CK_CNT}}{f_{PWM}}$$

f_{CK_CNT}	f_{PWM}	Steps ($ARR + 1$)
16 MHz	1 kHz	16 000 (~14 bits)
16 MHz	20 kHz	800 (~9.6 bits)
1 MHz	20 kHz	50 (~5.6 bits)

Choose the PWM frequency first (load needs), then PSC/ARR for the best resolution.

Edge-Aligned vs. Center-Aligned





- **Edge-aligned:** pulses all start together; simplest, used for most tasks
- **Center-aligned:** pulses are symmetric around the middle of the period; the frequency is half for the same ARR; preferred for motor drives (less harmonic content, easier current sampling)

Configuring PWM in C

Step 1: Clock and Pin (Alternate Function)

```
// Example: LD2 (PA5) is TIM2_CH1 (AF1) on the Nucleo-F411RE
RCC->AHB1ENR |= (1U << 0);          // GPIOA clock
RCC->APB1ENR |= (1U << 0);          // TIM2 clock

GPIOA->MODER &= ~(3U << (5*2));
GPIOA->MODER |=  (2U << (5*2));     // PA5: alternate function
GPIOA->AFR[0] |= (1U << (5*4));     // AF1 = TIM2_CH1
```

- The timer output reaches the pin only through the **alternate-function** mux of the GPIO
- Check the datasheet's alternate-function table for each pin

Step 2: Timer Registers

```
TIM2->PSC  = 15;                          // 16 MHz / 16 = 1 MHz
TIM2->ARR  = 999;                         // 1 kHz
TIM2->CCR1 = 250;                         // 25 % duty
TIM2->CCMR1 |= (6U << 4) | (1U << 3);     // OC1M = PWM mode 1, OC1PE
TIM2->CCER  |= (1U << 0);                 // CC1E: output enable
TIM2->EGR   |= (1U << 0);                 // UG: load shadow registers
TIM2->CR1   |= (1U << 0);                 // CEN: start counter
```

- To change the duty at run time, just write `TIM2->CCR1`
- **OC1PE** (preload) makes the new value take effect at the next period: no glitches

The Same with the HAL

```
htim2.Init.Prescaler = 15;            // 16 MHz / 16
htim2.Init.Period    = 999;           // ARR
sConfigOC.OCMode     = TIM_OCMODE_PWM1;
sConfigOC.Pulse      = 250;           // CCR1 -> 25 %

HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

// later: change the duty (0 .. ARR+1)
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 500);
```

- Same hardware, same registers: the HAL just writes them for you
- CCR = ARR + 1 gives 100 % duty in PWM mode 1

Common Mistakes

- Forgetting to enable the **timer clock** (RCC) or the **GPIO alternate function**
- Forgetting to set **CCxE** (output enable) or **CEN** (counter enable)
- Using the wrong prescaler: $f_{CK_CNT} = f_{CK_INT}/(PSC + 1)$, not $/PSC$
- Using a PWM frequency that is too low for the load (LED flicker) or too high for the resolution
- Driving a motor or large LED *directly* from the pin: use a driver (see the motors deck)
- Assuming 16-bit counters everywhere: TIM2 and TIM5 are 32-bit; TIM3 and TIM4 are 16-bit

Closing

Key Takeaways

- PWM approximates an **analog value** with a fast digital signal: $V_{avg} = D \cdot V_{+}$
- It is efficient because the switch is either fully ON or fully OFF
- In the STM32 timer: **PSC** divides the clock, **ARR** sets the period, **CCRx** sets the duty
- $f_{PWM} = f_{CK_INT}/((PSC + 1)(ARR + 1))$ and $D = CCR/(ARR + 1)$
- Higher frequency means lower resolution at the same clock
- The signal reaches the pin through the GPIO **alternate function**
- PWM + RC filter = simple DAC; PWM + driver = motor speed control